

Dokumentacja techniczna projektu

**Dokumentacja projektu grupowego
ID-346 - Analiza bio-sygnałów do identyfikacji osób**

Mikołaj Kuźmich 198291, Jakub Dąbrowski 197629, Mikołaj Jaworski 197636

1 Konwencje

1.1 Metoda pomiarowa

Dla kamizelki BRV wyznaczono następujący sposób dostosowania poziomu pasów:

- pas 1 - pod pachami
- pas 2 - na linii sutków
- pas 3 - na poziomie najniższego z żeber
- pas 4 - na poziome pępka
- pas 5 - na poziomie pasa

Wymaga się, by badany na potrzeby badania siedzącego, zajmującego około 15 minut nie wykonywał następujących czynności:

1.2 Struktura bazy danych

W celu usprawnionego przechowywania informacji zebranych w trakcie realizacji projektu grupowego utworzono bazę danych MongoDB. Baza danych zawiera podstawowo 2 kolekcje:

- segmenty badań składające się z pomiarów - **segments**

Pole	Typ	Opis
sourceMeasurementDate	timestamp	data wykonania pomiaru źródłowego
measurementLength	int	długość pomiaru w minutach
subject	string	PESEL osoby badanej
sampleArray	Sample[]	tablica próbek stanowiących jeden segment pomiaru

- wyekstrahowane cechy z tych segmentów - **features**

2 Dane

2.1 Obiekt pomiaru

2.2 Etapy przetwarzania danych

2.2.1 Obsługa danych wejściowych i wstępne przetwarzanie

Pierwszym etapem przetwarzania danych jest ich wczytanie oraz odpowiednie sformatowanie. Dane, zgodnie z opisem w sekcji 1, są przechowywane w plikach `.jsonl`. Każda linia reprezentuje pojedynczą próbkę zarejestrowaną przez kamizelkę pomiarową.

Wczytywanie odbywa się sekwencyjnie. Każda linia zawiera surowe dane czasowe (godziny, minuty, sekundy, milisekundy), 24-bitowe wartości z pięciu przetworników ADC (podzielone na trzy 8-bitowe składowe) oraz dane z czujników inercyjnych IMU. Poniżej przedstawiono przykładowy format surowej próbki:

```
"hrs": 0, "min": 0, "sec": 13, "ms": 457,
"adc1_output_23_to_16": 240, "adc1_output_15_to_8": 8,
"adc1_output_7_to_0": 165, "adc2_output_23_to_16": 255,
"adc2_output_15_to_8": 36, "adc2_output_7_to_0": 15,
"adc3_output_23_to_16": 243, "adc3_output_15_to_8": 99,
"adc3_output_7_to_0": 199, "adc4_output_23_to_16": 253,
"adc4_output_15_to_8": 161, "adc4_output_7_to_0": 55,
"adc5_output_23_to_16": 238, "adc5_output_15_to_8": 46,
"adc5_output_7_to_0": 1, "imu1_output_acc_x": 2384,
"imu1_output_acc_y": 237, "imu1_output_acc_z": 16526,
"imu1_output_gyr_x": -12, "imu1_output_gyr_y": -11,
"imu1_output_gyr_z": 17, "imu2_output_acc_x": 3021,
"imu2_output_acc_y": 888, "imu2_output_acc_z": 16327,
"imu2_output_gyr_x": -1, "imu2_output_gyr_y": -31,
"imu2_output_gyr_z": 17, "imu3front_output_acc_x": 5654,
"imu3front_output_acc_y": 866, "imu3front_output_acc_z": 15351,
"imu3front_output_gyr_x": -2, "imu3front_output_gyr_y": 7,
"imu3front_output_gyr_z": -5, "imu3back_output_acc_x": -831,
"imu3back_output_acc_y": -2094, "imu3back_output_acc_z": 16238,
"imu3back_output_gyr_x": 2, "imu3back_output_gyr_y": 9,
"imu3back_output_gyr_z": -9, "imu4_output_acc_x": 1997,
"imu4_output_acc_y": 655, "imu4_output_acc_z": 15972,
"imu4_output_gyr_x": -5, "imu4_output_gyr_y": -4,
"imu4_output_gyr_z": -2, "imu5_output_acc_x": -768,
"imu5_output_acc_y": 437, "imu5_output_acc_z": 16111,
"imu5_output_gyr_x": -3, "imu5_output_gyr_y": 3,
"imu5_output_gyr_z": 7, "output_status_adc1": true,
"output_status_adc2": true, "output_status_adc3": true,
"output_status_adc4": true, "output_status_adc5": true,
"output_status_imu1": true, "output_status_imu2": true,
"output_status_imu3front": true, "output_status_imu3back": true,
"output_status_imu4": true, "output_status_imu5": true,
"output_status_padding": 0
```

W celu ułatwienia dalszej obróbki, każda z tych linii jest parsowana do wewnętrznej struktury danych. Proces ten obejmuje połączenie trzech 8-bitowych rejestrów każdego z pięciu kanałów ADC w jedną wartość liczbową, która następnie jest przeliczana na napięcie (wolty) zgodnie ze specyfikacją zastosowanych tensometrów.

Struktura danych wynikowych

Po wstępnym przetworzeniu dane są normalizowane i zapisywane w formie uproszczonych obiektów JSON, które stanowią podstawową jednostkę danych w dalszych etapach projektu. Przykładowy format przetworzonego segmentu (np. 2-minutowego) wygląda następująco:

```
{
  "timestamp": 0,
  "adc_outputs": [
    0.0002464437,
    0.0003881085,
    0.0004560553,
    -1.75746999e-05,
    0.0002706136
  ]
}
```

Tu warto dodać, że dla większości obliczeń wykonywanych w dalszych etapach takich jako wyciąganie cech, separacja oddechów czy wykrywanie anomalii pracujemy na sygnale dla środkowego ADC, eksperymentalnie ustaliliśmy, że jest on najbardziej miarodajny w kontekście analizy oddechu, a praca z jednym czujnikiem znacznie upraszcza cały proces.

2.2.2 Oddzielanie oddechów

Celem tego etapu jest wstępne podzielenie sygnału na mniejsze części reprezentujące kolejne oddechy w analizowanym pomiarze. Realizowane jest to poprzez wyznaczenie wszystkich maksymów i minimów lokalnych oraz zapisanie ich w programie tak, by przygotować je do dalszej obróbki.

- **Dane wejściowe:**

Zbiór wartości odstawiania od średniej wartości sygnału na danym pasie, dla każdego pasa w kamizelce pomiarowej. Każda z tych wartości przypisany ma odpowiedni znacznik czasu od rozpoczęcia pomiaru, w którym pobrano próbkę.

- **Dane wyjściowe:**

Zbiór punktów reprezentujących wszystkie maksyma i minima lokalne sygnału wraz z przypisanymi im wartościami znaczników czasu.

2.2.3 Wykrywanie anomalii

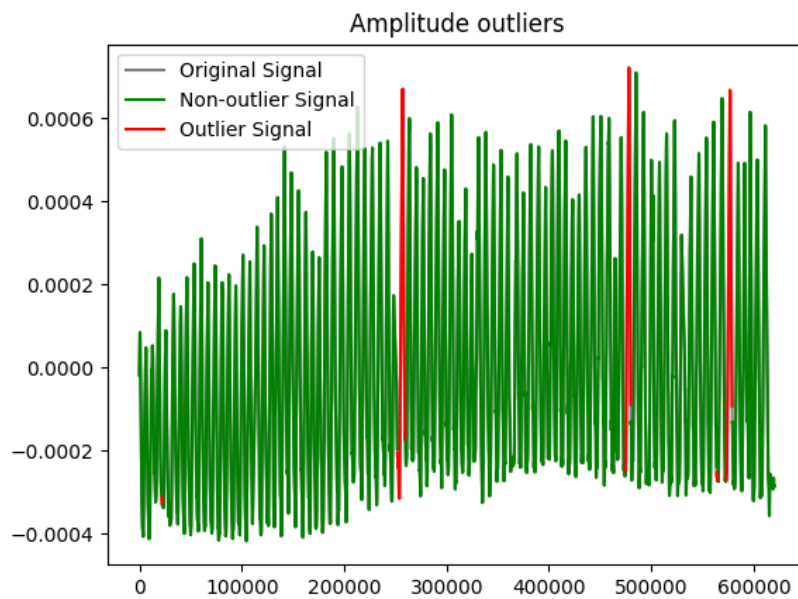
Kolejnym kluczowym etapem jest identyfikacja oraz eliminacja anomalii (ang. *outlier detection*). Sygnał z czujników tensometrycznych w kamizelce jest podatny na zakłócenia wynikające z nagłych ruchów użytkownika, poprawiania kamizelki czy chwilowych błędów transmisji danych. W ten sposób mogą pojawić się oddechy o nietypowych wartościach, które mogą negatywnie wpłynąć na jakość analizy i klasyfikacji.

Proces ich wykrywania i usuwania opiera się na analizie statystycznej zbioru wyznaczonych wcześniej ekstremów (maksymów i minimów) oraz odległości czasowych między nimi.

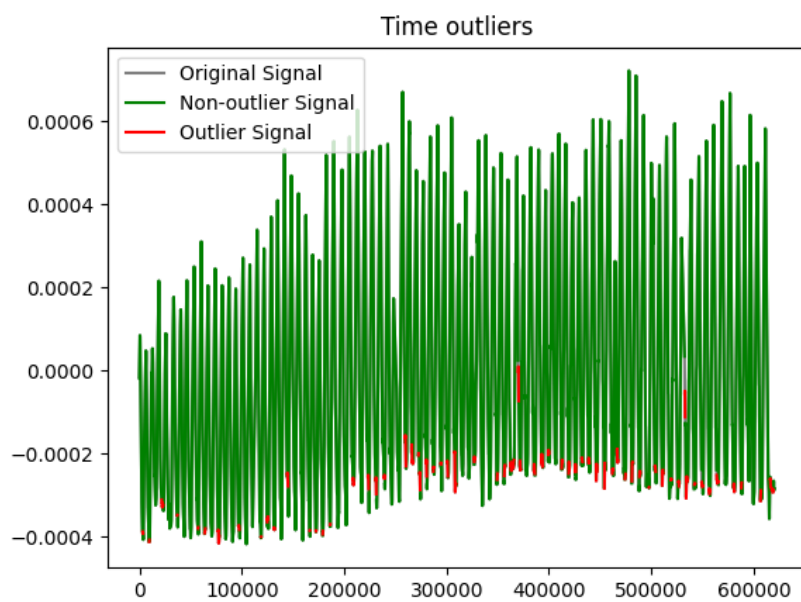
- **Kryteria wykrywania anomalii:**

- **Amplituda:** Odrzucane są oddechy o wartościach należących do górnych i dolnych kwartyli rozkładu amplitud. Obecnie odrzucamy około jeden procent oddechów z każdej strony rozkładu.

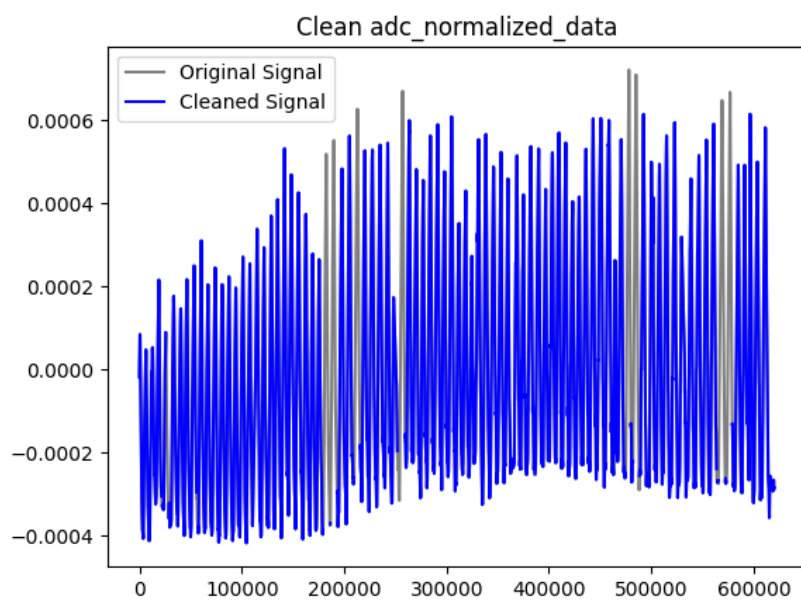
- **Częstotliwość:** Odrzucane są oddechy o zbyt krótkich oraz długich odległościach między wykrytymi wcześniej maksymami. Obecnie odrzucamy około jeden procent oddechów z każdej strony rozkładu.
- **Dane wejściowe:**
Zbiór par (znacznik czasu, wartość) dla zidentyfikowanych maksimów i minimów lokalnych z poprzedniego etapu oraz znormalizowany sygnał oddechowy.
- **Dane wyjściowe:**
Przefiltrowany zbiór punktów (zmodyfikowany sygnał oddechowy), o odpowiednio przesuniętych wartościach znaczników czasu.



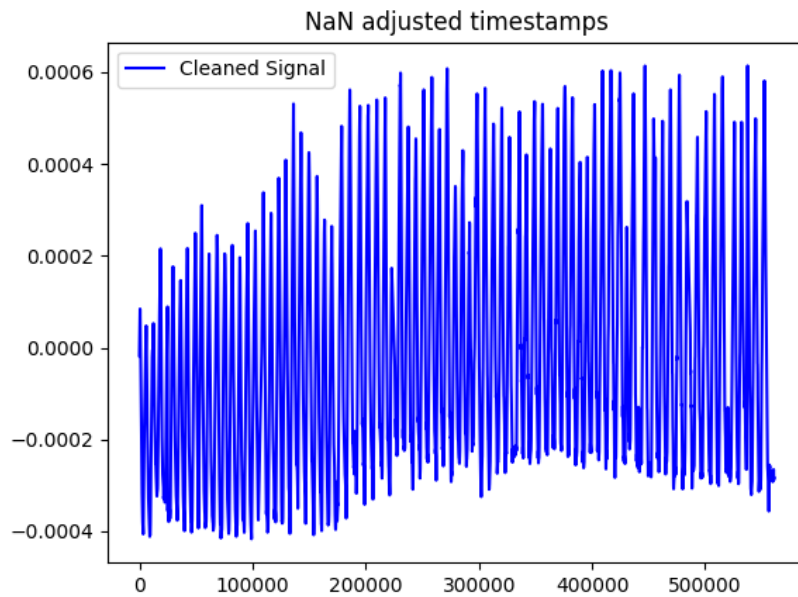
Rysunek 2.1: Wykryte anomalie na podstawie amplitudy dla przykładowego sygnału.



Rysunek 2.2: Wykryte anomalie na podstawie Częstotliwości dla przykładowego sygnału.



Rysunek 2.3: Wykryte anomalie po usunięciu artefaktów dla przykładowego sygnału.



Rysunek 2.4: Odtworzony sygnał po usunięciu anomalii dla przykładowego sygnału.

2.2.4 Podział danych na segmenty i resampling

Ostatnim krokiem przygotowania danych przed ekstrakcją cech jest podział przefiltrowanego sygnału na okna czasowe (segmenty) o stałej długości dwóch minut oraz wykonanie resamplingu.

Najpierw sygnał jest resamplowany do za pomocą interpolacji liniowej (funkcja `interp1d` z biblioteki SciPy) do Częstotliwości 10 Hz, a następnie dzielony na segmenty. Dzięki temu każdy segment zawiera około 1200 próbek. Sam algorytm podziału na segmenty jest raczej prosty - rozpoczynamy od początku sygnału i zapisujemy do odpowiednio nazwanych plików `.jsonl` kolejne próbki danych o długości dwóch minut.

2.2.5 Ekstrakcja cech

Po podzieleniu danych na segmenty, a więc ostatecznym przygotowaniu ich do analizy, należy rozpocząć proces pozyskiwania informacji potrzebnych do prawidłowego działania klasyfikatora. Sama sieć neuronowa nie potrzebuje zdefiniowanych cech sygnału, by funkcjonować, jednak w celach badawczych, zdecydowano się na ich ekstrakcję ze względu na możliwość sprawdzenia różnic we wzorcach oddechowych objętych analizą osób.

- **Dane wejściowe:**

Zbiór plików zawierających podzielone na segmenty dane z pomiarów po wszystkich przekształceniach, w których nazwa wyznacza etykietę, ponieważ zawiera typ aktywności oraz identyfikator osoby

- **Dane wyjściowe:**

Plik wyjściowy `extracted_features.jsonl` w formacie `.jsonl`, zawierający wektory cech wyznaczone dla kolejnych segmentów danych pomiarowych.

Cechy składające się na wektor dla segmentu:

- **bpm** - liczba oddechów na minutę
- **breath_depth** - wyznaczone poprzez uśrednienie wartości szczytów oddechów w całym segmencie dla środkowego pasa

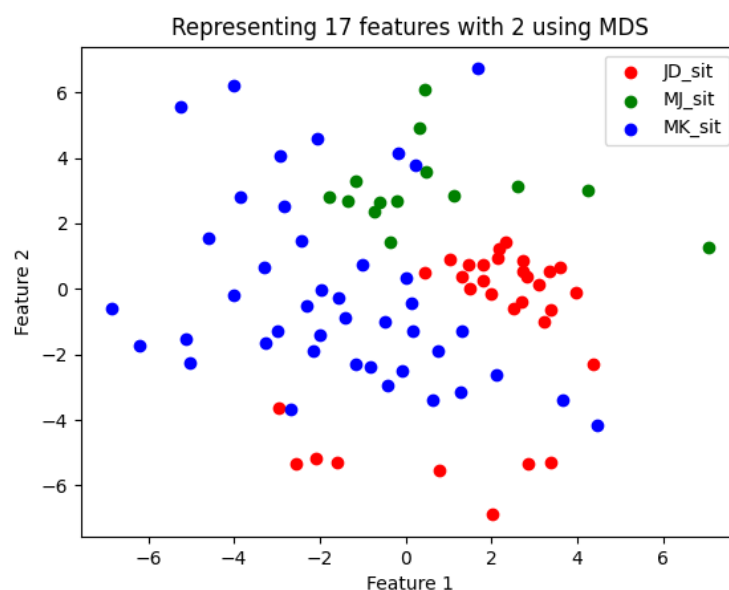
- **breath_depth_std** - odchylenie standardowe sygnału, jako głębokość oddechu dla środkowego pasa
- **belt_share** - udział poszczególnych pasów w oddychaniu, reprezentowany przez 5 wartości, po jednej dla każdego pasa. Jest on wyznaczany poprzez zsumowanie wartości głębokości oddechu dla każdego pasa, tworzy to mianownik ułamka, który liczony jest osobno dla każdego tensometru, gdzie licznikiem jest głębokość oddechu dla tego konkretnego sensora
- **belt_share_std** - wyznaczany jest on analogicznie do powyższego, korzysta jednak z wyżej wspomnianych odchyśleń standardowych sygnału, jako głębokości
- **breathing_phase_lengths** - jest to wartość średnia czasu trwania kolejnych faz oddychania, a mianowicie: faza wdechu, pauza po wdechu, faza wydechu, pauza po wydechu

2.3 Wizualizacja danych

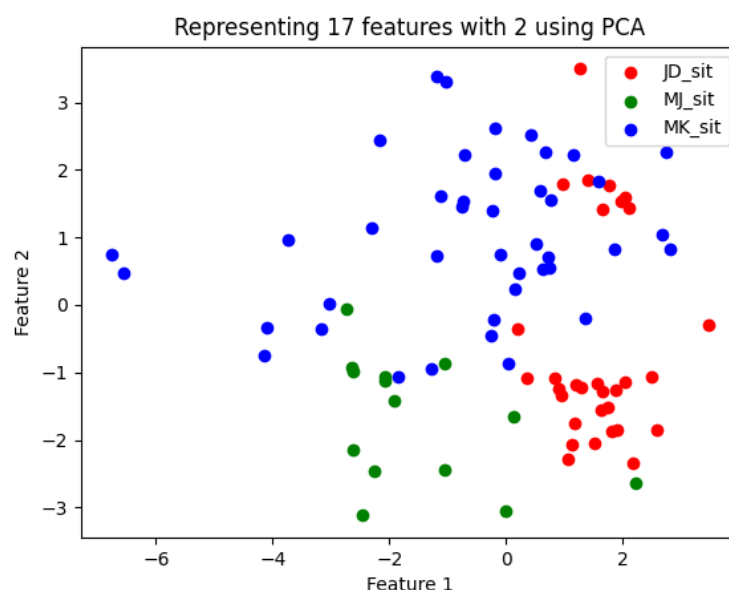
Aby zweryfikować poprawność całego procesu przetwarzania należało w odpowiedni sposób zwizualizować dane pozyskane w ramach projektu. Jest to niezbędny etap, gdyż pozwala na ewaluację postępów.

Trzy główne cele etapu to:

1. **Weryfikacja rozróżnialności osób po przetwarzaniu** - Aby móc zweryfikować poprawność procesu przetwarzania, trzeba zwizualizować zebrane dane. Jest to realizowane za pomocą algorytmów redukujących liczbę cech: **MDS** oraz **PCA**. W tym podpunkcie liczba składowych, w celach poglądowych, obniżana jest do dwóch. Dzięki tym wartościom jesteśmy w stanie wyznaczyć kierunek, będący transformacją macierzy danych, który spowoduje największe możliwe zróżnicowanie zbioru danych. Zakładając więc, że różnice we wzorcach oddechowych między różnymi osobami będą stosunkowo dużo większe niż między różnymi momentami dla tej samej osoby, jesteśmy w stanie uzyskać odpowiednią, wizualną separację danych, zgodną z etykietami.
2. **Wyznaczenie najbardziej istotnych cech dla klasyfikatora** - W celu wyżej wspomnianej weryfikacji przetwarzania danych, należało zredukować liczbę wymiarów wektora cech, tak by ten mógł zostać przedstawiony w sposób możliwy do analizy przez człowieka. Zostały więc wyznaczone w ten sposób najbardziej istotne elementy sygnału. Dzięki temu jesteśmy w stanie zredukować ostateczną ich liczbę, co pozytywnie wpływa na ryzyko zbytniego dopasowania oraz pozwala na mniejszy nakład obliczeniowy związany np. ze skorelowanymi zmiennymi.



Rysunek 2.5: Cechy po wywołaniu algorytmu MDS przedstawione na dwuwymiarowym wykresie.



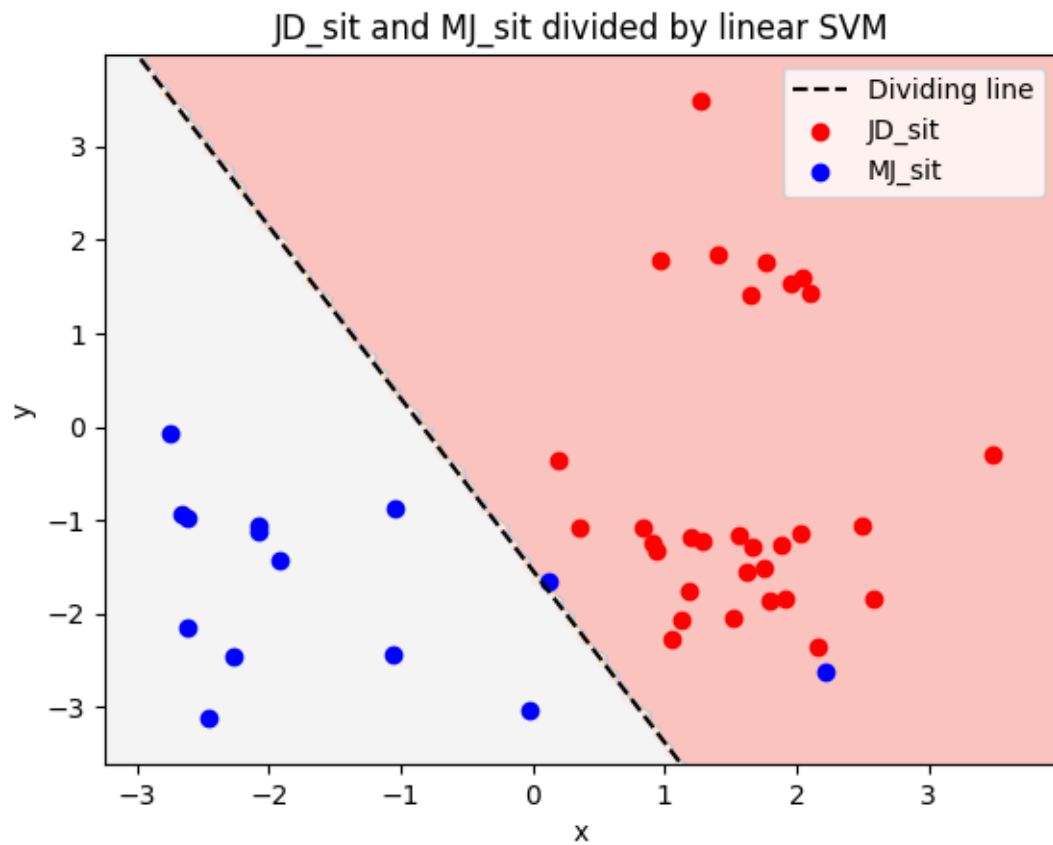
Rysunek 2.6: Cechy po wywołaniu algorytmu PCA przedstawione na dwuwymiarowym wykresie.

3. **Kontrola jakości przez prosty klasyfikator** - Ostatnim celem Wizualizacji jest kontrola jakości rozwiązania dzięki klasyfikatorowi. Skorzystano więc z prostego klasyfikatora binarnego: **SVM**, który wykorzystywany jest do porównania zbiorów danych o dwóch różnych etykietach. Przy okazji wizualizacji wyznaczana jest też wartość liczbową dokładności takiego klasyfikatora. Użycie tego konkretnego algorytmu pozwoli na podkreślenie ewentualnych problematycznych par podobnych zbiorów danych. Dodatkowo, dokonywana jest również ewaluacja za pomocą sieci **LSTM**.

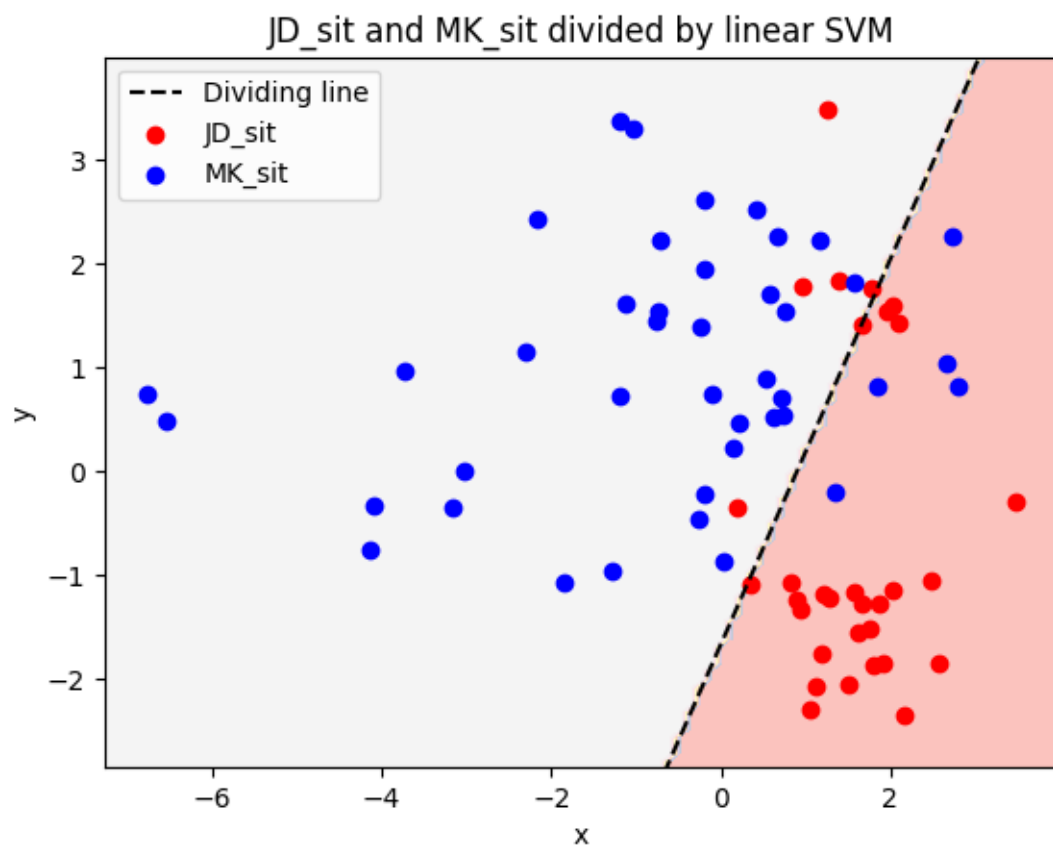
Opis przeprowadzonej ewaluacji

2.3.1 SVM - Support Vector Machine

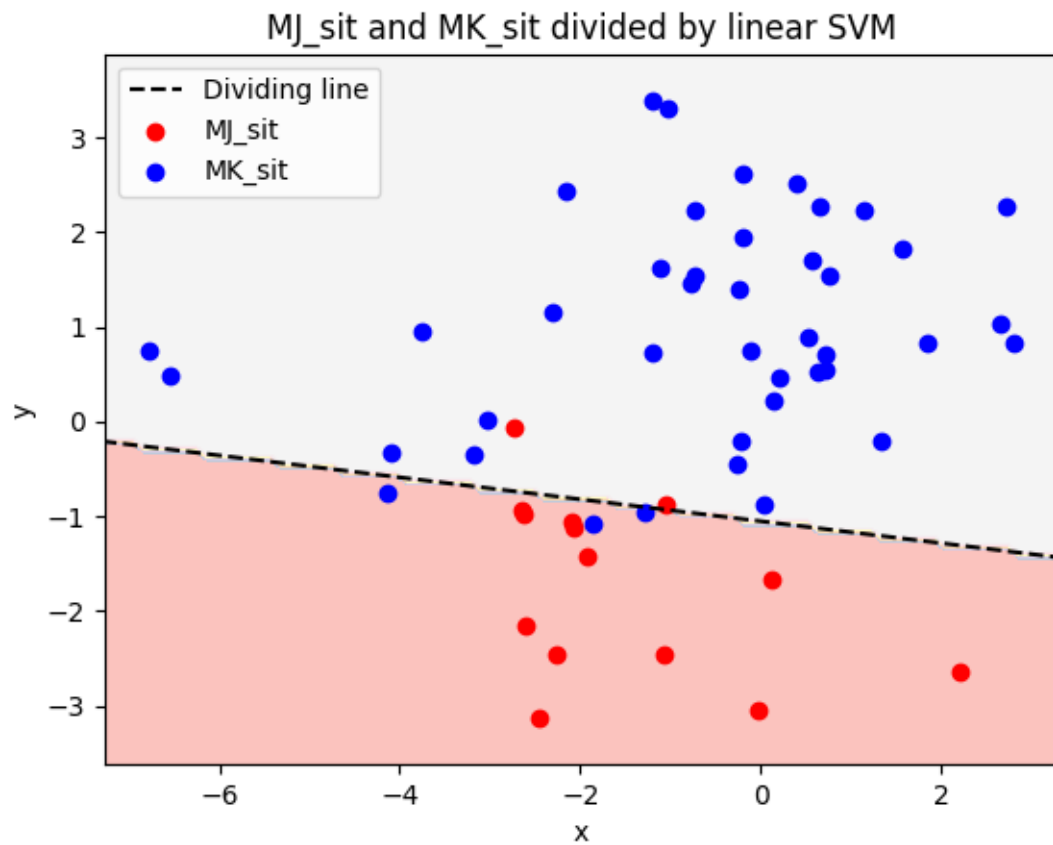
Za pomocą klasyfikatora SVM, jesteśmy w stanie podzielić zbiór na dwa, poprzez znalezienie odpowiedniej hiperpłaszczyzny maksymalizującej margines między dwoma klasami. W związku z tym, że jest on klasyfikatorem binarnym, należało wywołać go osobno dla każdej możliwej pary etykiet w ogólnym zbiorze zredukowanych cech. W ten sposób uzyskujemy podział dla wszystkich kombinacji oraz związaną z nimi dokładność. Poniżej przykłady dla zbioru danych 3 osób.



Zmierzona dokładność: Accuracy for JD_sit and MJ_sit: 95.56%



Zmierzona dokładność: Accuracy for JD_sit and MK_sit: 84.93%



Zmierzona dokładność: Accuracy for MJ_sit and MK_sit: 91.07%

2.3.2 LSTM - Long Short-Term Memory

not yet

2.4 Kod źródłowy

Struktura kodu źródłowego została zaprojektowana w sposób modularny, co pozwala na separację logiki przetwarzania sygnałów, ekstrakcji cech oraz wizualizacji rezultatów. Całość implementacji podzielono na trzy główne segmenty funkcjonalne.

2.4.1 Architektura systemu

Kod realizuje pełny potok przetwarzania danych (pipeline) – od surowych odczytów z sensorów kamizelki, aż po wizualizację wyników klasyfikacji. Główne moduły to:

- **Przetwarzanie wejścia (Input Processing):** Obsługa plików JSONL, czyszczenie danych (anomalie), wygładzanie spline-ami oraz segmentacja na 2-minutowe interwały.
- **Wykrywanie cech (Feature Extraction):** Przekształcenie sygnału czasowego na wektory cech (np. BPM, głębokość oddechu, fazy oddechu).
- **Wizualizacja danych:** Implementacja algorytmów redukcji wymiarowości (PCA, MDS) oraz ewaluacja separowalności klas przy użyciu SVM.

2.4.2 Kluczowe funkcje i przepływ danych wewnątrz skryptu main

Przepływ danych opiera się na klasie `ADC_data`, która przechowuje obrabiane dane na różnych etapach przetworzenia, w tym znormalizowane wyniki pomiarów w voltach. Poniżej przedstawiono kluczowe funkcje procesowe:

1. `process_file()` oraz `handle_input_data()`: Odpowiadają za parsowanie surowego formatu JSON (zawierającego dane z akcelerometrów IMU i tensometrów ADC) do ustandaryzowanych obiektów procesowych.
2. `breath_separation()`: Wykorzystuje analizę ekstremów lokalnych do podziału ciągłego sygnału na pojedyncze cykle oddechowe.
3. `outlier_detection()`: Realizuje filtrację statystyczną, usuwając $x\%$ oddechów o najbardziej odbiegających parametrach czasowo-amplitudowych, co minimalizuje wpływ błędów pomiarowych.
4. `split_data_into_segments()`: Normalizuje długość próbek poprzez resampling i dzieli sygnał na fragmenty ułatwiające dalszą analizę statystyczną.

2.4.3 Kluczowe funkcje i przepływ danych wewnątrz skryptu `extract_features`

Skrypt ten odpowiada za wyznaczenie wektorów cech dla segmentów oczyszczonego sygnału stanowiącego wyjście poprzedniego skryptu. Realizuje on to przez wybranie segmentów, których procent utraconych danych nie przekroczył wartości `ACCEPTABLE_DATA_LOSS` ustawionej bazowo na 0.2

- `basic_feature_extraction()`: Funkcja, w której środku następuje ekstrakcja wektora cech. Jest ona wywoływana pojedynczo, dla każdego segmentu analizowanego przez skrypt (mniej niż 20% utraty danych). Procedury wykonywane w ramach tej funkcji zostały opisane w sekcji 2.2.5.

2.4.4 Kluczowe funkcje i przepływ danych wewnątrz skryptu `visualize_data`

Skrypt ten odpowiada za wizualizację danych po przetworzeniu i ekstrakcji cech. Kluczowe funkcje i elementy stanowiące część wizualizacji to:

- `NO_OF_FEATURES_AFTER_ALG`: Stała mówiąca o tym, do ilu cech należy zredukować nasze wektory.
- `FeatureData`: Struktura trzymająca dane zarówno przed, jak i po redukcji cech, etykiety dla danych oraz kolory do wizualizacji.

- **feature_loading**: Funkcja zajmująca się wczytaniem wektorów cech z pliku `extracted_features.jsonl` do wyżej wspomnianej struktury.
- **MDS_algorithm**: Funkcja wywołująca algorytm redukcji cech MDS i wyświetlająca jej wynik na ekranie wraz z etykietami danych.
- **PCA_algorithm**: Funkcja wywołująca algorytm redukcji cech PCA i zapisująca wynik w strukturze.
- **SVM_validation**: Funkcja mająca na celu walidację możliwości podziału zbioru binarnie przez hiperpłaszczyznę.
- **plotheatmap**: Funkcja standaryzująca dane do przedziału $[0,1]$ (dla każdej cechy osobno) i wyświetlająca je za pomocą heatmapy.

2.4.5 Kluczowe funkcje i przepływ danych wewnątrz skryptu `create_data_profiles`

Skrypt ten odpowiada za tworzenie indywidualnych profili oddechowych każdej osobie, dla której zostały zebrane dane. Dzięki temu możliwe jest przedstawienie i dostrzeżenie indywidualnych cech oddechowych każdej osoby i wizualne potwierdzenie zasadności przeprowadzanego badania. Każdy taki profil jest Pythonową mapą i zawiera:

- **signal**: Jest to wycinek surowego sygnału oddechowego danej osoby zawierający jeden oddech, który statystycznie najlepiej reprezentuje jej sposób oddychania ze względu na szereg parametrów stanowiących kolejne pola tej mapy.
- **timestamps**: Odpowiada za przechowywanie znaczników czasowych odpowiadających kolejnym punktom sygnału.
- **inhale, inspiratory_phase, exhale, expiratory_phase**: Cztery pola odpowiadające kolejnym fazom oddechu. Każde z nich zawiera różnicę w timestampach pomiędzy początkiem i końcem danej fazy.
- **length**: Długość całego oddechu w milisekundach.
- **depth**: Głębokość oddechu, czyli różnica pomiędzy maksymalną a minimalną wartością sygnału w danym oddechu.

Do zadań na przyszłość należy rozszerzenie tej mapy o dodatkowe pola zawierające cechy, które dokładniej pozwolą charakteryzować indywidualny sposób oddychania każdej osoby. Poniżej znajdują się najważniejsze funkcje odpowiadające za tworzenie profili oddechowych:

- **create_data_profiles()**: Główna funkcja tworząca profile oddechowe dla każdej osoby na podstawie przetworzonych danych.
- **get_people_list()**: Funkcja zwracająca listę osób na podstawie etykiet danych znajdujących się w folderze `results`.
- **calculate_breath_characteristics()**: Oblicza fazy oddechu (wdech, faza inspiracji, wydech, faza ekspiracji) na podstawie sygnału dla każdego wykrytego oddechu.
- **get_mode_breath()**: Wybiera najczęściej występujący oddech na podstawie cech statystycznych.
- **visualize_profiles()**: Tworzy wykresy przedstawiające profile oddechowe każdej osoby.